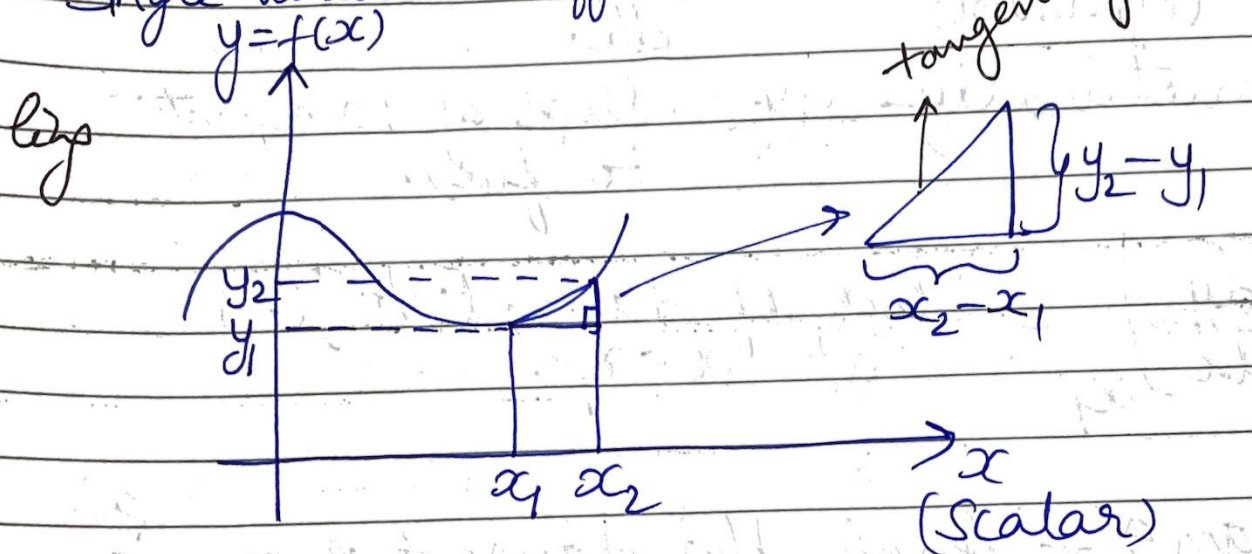


Ch 10: Solving Optimization Problems

VI: Differentiation

Single-variable differentiation



$$\frac{dy}{dx} = \frac{df}{dx} = y' \text{ or } f'$$

↑ differentiation of y wrt x

$\frac{dy}{dx} \rightarrow$ how much does y change as x changes.

$$\frac{y_2 - y_1}{x_2 - x_1} = \frac{\Delta y}{\Delta x} = \tan \theta$$

$$\frac{dy}{dx} = \lim_{\Delta x \rightarrow 0} \frac{\Delta y}{\Delta x}$$

as change in x is very very small.

tangent of $f(x)$ = line that touches the point ~~at~~, it is the hypotenuse when $\Delta x \rightarrow 0$

As $\Delta x \rightarrow 0$, $\frac{\Delta y}{\Delta x}$ becomes $\frac{dy}{dx}$.

So $\tan \theta$ becomes $\frac{dy}{dx}$.
(hypotenuse)

Geometrically $\frac{dy}{dx} = \text{Slope of tangent to } f(x)$.

$\left[\frac{dy}{dx} \right]_{x_1} = \text{Slope of tangent to } f(x) \text{ at } x = x_1$.

$\tan \theta = \text{Angle b/w tangent \& } x\text{-axis}$

$$1) \frac{d x^n}{dx} = n x^{n-1}$$

$$2) \frac{d(c)}{dx} = 0$$

$$3) \frac{d(x^n)}{dx} = n x^{n-1}$$

$$4) \frac{d \log(x)}{dx} = \frac{1}{x}$$

$$5) \frac{d(e^x)}{dx} = e^x$$

$$6) \frac{d}{dx} (f(x) + g(x)) = \frac{d(f(x))}{dx} + \frac{d(g(x))}{dx}$$

Chain Rule

$$\boxed{\frac{d f(g(x))}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}}$$

$$Q \quad f(g(x)) = (a - bx)^2$$

$$\text{So, } g(x) = (a - bx), \quad f(x) = x^2$$

$$\text{Ans } \frac{d(f(g(x)))}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

$$\frac{dg}{dx} = (a - bx) \Rightarrow \frac{d}{dx} (a) - \frac{d(bx)}{dx}$$

$$\Rightarrow -b$$

$$\frac{df}{dg} = \frac{d(x^2)}{dx} = 2x$$

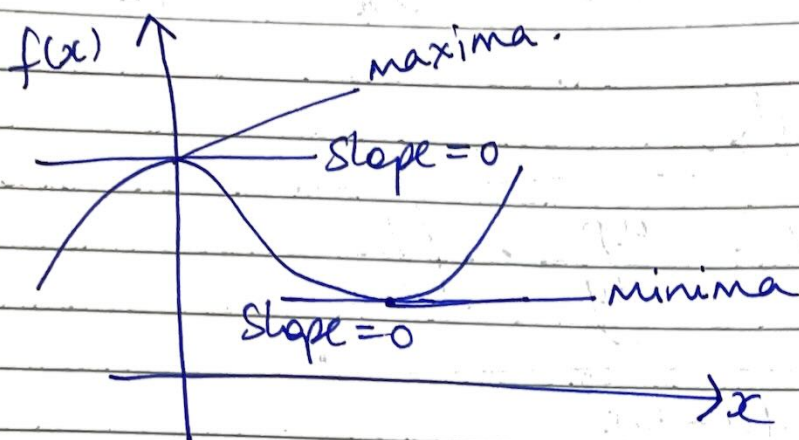
$$\frac{df}{dg} \cdot \frac{dg}{dx} = -2b(a - bx)$$

~~forget this~~

V2: Online derivative calculators:-

- 1) derivative-calculator.net
- 2) symbolab.com
- 3) wolframalpha.com (plots)

V3: Maxima and Minima



$$f(x) = x^2 - 3x + 2$$

At minima & maxima
slope = 0.

maxima?
minima?

$$\text{Slope} = \frac{df}{dx} = 0.$$

$$= \frac{d(x^2 - 3x + 2)}{dx} = 2x - 3$$

$$\text{At } x = \frac{3}{2}, \text{ Slope} = 0.$$

$$f(1.5) = -0.25 \text{ cannot be maxima.}$$

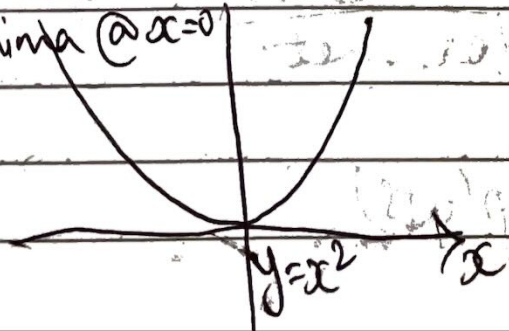
$$f(1) = 0.$$

$$f(1.5) < f(1)$$

So ~~is~~ minima @ $x = 1.5$

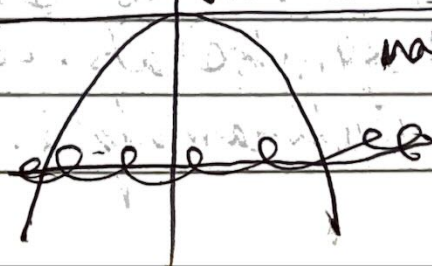
no maxima.

minima @ $x=0$

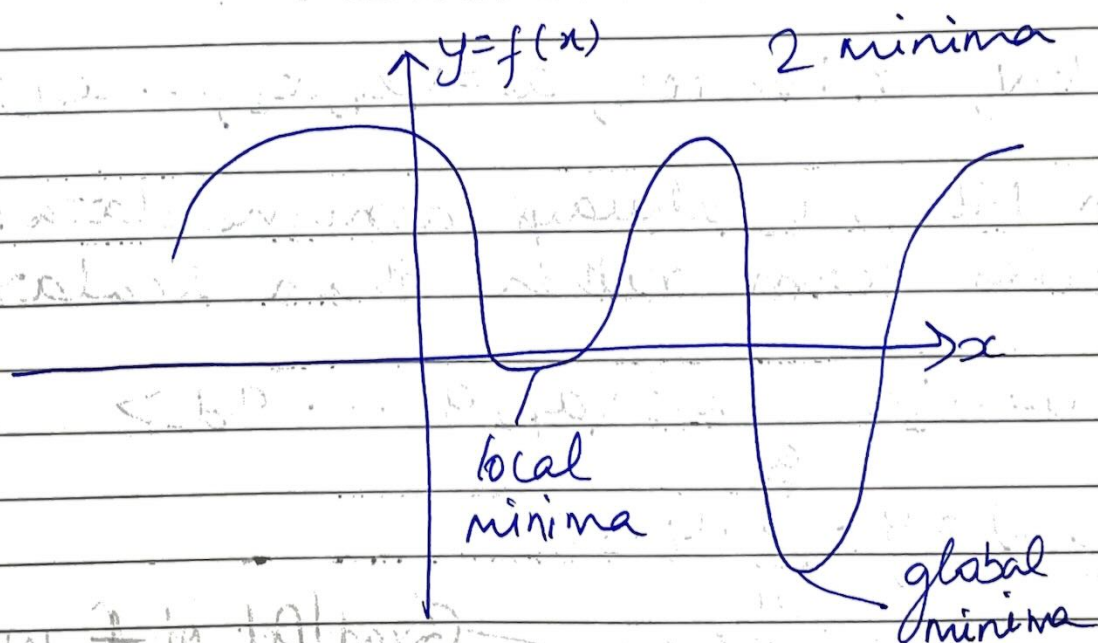
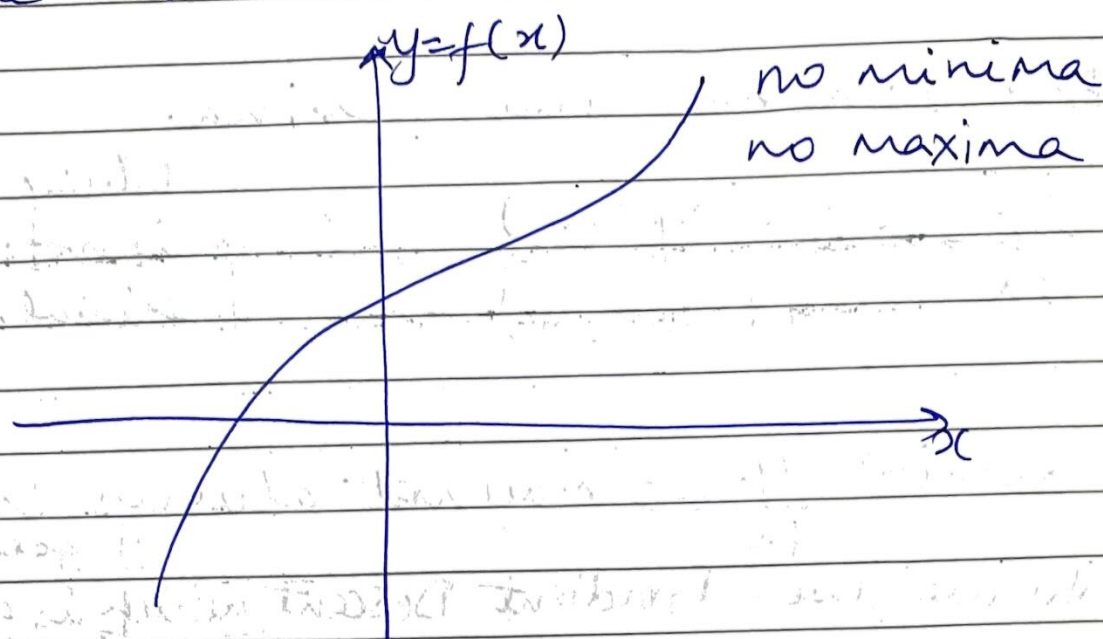


$$f(x) = y$$

maxima @ 0



Some function may/not maxima or minima. or may have more than one minima or maxima



A function can have multiple minima or maxima.

minimum of all minima is global minima
only for global maxima.

$$f(x) = \log(1 + \exp(ax)) \quad \text{— like logistic loss}$$

Find minima and maxima.

$$\frac{df}{dx} = \frac{a \exp(ax)}{1 + \exp(ax)} = 0 \quad \left. \vphantom{\frac{df}{dx}} \right\} \begin{array}{l} \text{solving this} \\ \text{is not} \\ \text{trivial.} \end{array}$$

Solving $\frac{df}{dx} = 0$ may not always be good.

So we use Gradient Descent which is a computer algo.

V4: Vector Calculus: Grad

Now, x : Vector, $x = \langle x_1, x_2, \dots, x_d \rangle$

In ML, we always assume data in Vector form rather than scalar.

$$y = a^T x \quad a = \langle a_1, a_2, \dots, a_d \rangle$$

$$f(x) = y = \sum_{i=1}^d a_i x_i$$

$$\frac{df}{dx} = \nabla_x f \quad \leftarrow \begin{array}{l} \text{Grad/Del of } f \text{ wrt } x \\ \text{grad or del} \\ \text{for vector differentiation} \end{array}$$

Vector

$$\nabla_x f = \begin{bmatrix} \frac{df}{dx_1} \\ \frac{df}{dx_2} \\ \vdots \\ \frac{df}{dx_d} \end{bmatrix} \rightarrow \text{vector} \in \mathbb{R}^d$$

$\frac{df}{dx_1}$ = partial differentiation
'dough'

$$y = a^T x = \sum_{i=1}^d a_i x_i$$

$$= a_1 x_1 + a_2 x_2 + \dots + a_d x_d$$

$$\nabla_x f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_d \end{bmatrix} = a$$

$$\text{So, } \nabla_x (a^T x) = a \Leftrightarrow \frac{d(a^T x)}{dx} = a$$

Logistic Loss
with Regularization

eg -

$$L(w) = \sum_{i=1}^n \log(1 + \exp(-y_i w^T x_i)) + \lambda w^T w$$

$\langle x_i, y_i \rangle \rightarrow \text{constants} \rightarrow D_{\text{train}}$

$$\nabla_w L = \frac{(-y_i x_i) (\exp(-y_i w^T x_i))}{1 + \exp(-y_i w^T x_i)} + 2\lambda w$$

grad of L
wrt w

= 0.

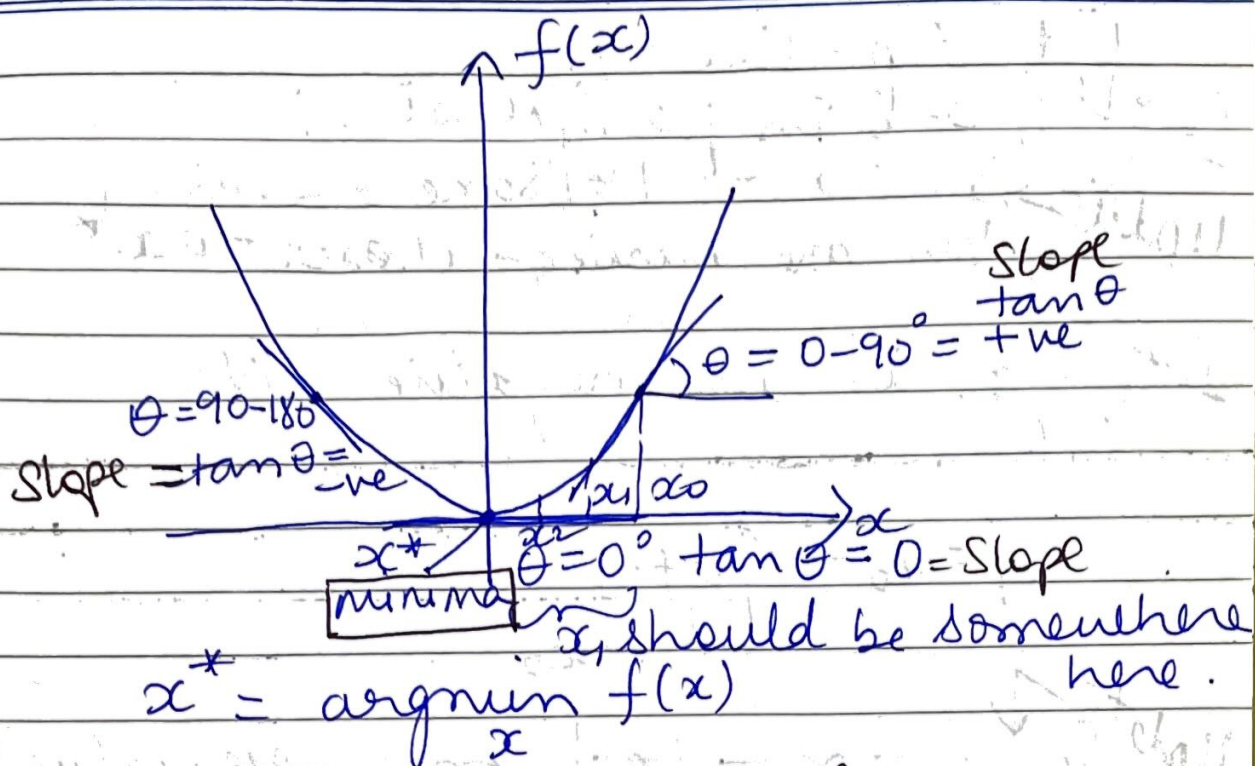
Very hard to solve.

VS: Gradient descent: Geometric Intuition.

It is an iterative algorithm. Easy to implement in a modern computer.

- 1) Pick x_0 as any random no. $\leftarrow$
- 2) $x_1 \leftarrow$ iteration 1. first guess
- 3) $x_2 \leftarrow$ " 2. of x^*
- $\vdots$
- $\vdots$
- $\vdots$
- k) $x_k \leftarrow$ " k (very close to x^*)

As iteration $\uparrow$ we move closer to x^*



Rules $\begin{cases} \min f(x) \Rightarrow \max(-f(x)) \\ \max f(x) \Rightarrow \min(-f(x)) \end{cases}$

At minima of this curve, $\tan \theta$ or slope ≤ 0 .

As you move from x_0 to x^* your slope decreases. and vice versa on the other side. At minima slope changes from +ve to -ve and -ve to +ve.

Algo:

- 1) Pick an initial point $= x_0$ @ random (Initialization point).
 - 2) $x_1 = x_0 - \gamma \left[\frac{df}{dx} \right]_{x_0}$ Finding x_1 closer to x^* than x_0 .
- γ $\downarrow$ step size (constant here)
let step size $= 1$

η is always +ve.

Let step-size = 1.

Case 1: If x_0 is in positive side :-

So $x_1 = x_0 - 1 * [\text{some +ve value}]$
Update: You are moving closer to x^*

Case 2: If x_0 is in negative side,
then

$$x_1 = x_0 - \eta \left[\frac{df}{dx} \right]_{x_0}$$

Update: $\downarrow$
So, $x_1 = x_0 - 1 * [\text{some -ve value}]$
= You are moving closer to x^*

So update func. works in both cases when x_0 is either +ve or -ve.

$$3) x_2 = x_1 - \eta \left[\frac{df}{dx} \right]_{x_1}$$

At any iteration.

$$\boxed{x_{i+1} = x_i - \eta \left[\frac{df}{dx} \right]_{x_i}} \quad \text{update function.}$$

$x_0, x_1, x_2, \dots, x_k$

$$x_k = x_{k-1} - \eta \left[\frac{df}{dx} \right]_{x_{k-1}}$$

(eventually) η is set to 0
1 = with new η

Now, $x_0, x_1, x_2 \dots x_k, x_{k+1}$

if $(x_{k+1} - x_k)$ is v. small

So, stop updating.

then terminate @ $x^* = x_k$

optimal value.

$$x_{i+1} = x_i + \gamma \left(\frac{df}{dx} \right)_{x_i}$$

Note $\rightarrow \left[\frac{df}{dx} \right]_{x_0} \geq \left[\frac{df}{dx} \right]_{x_1} \geq \left[\frac{df}{dx} \right]_{x_2} \geq \dots$

Initially you make a big jump from x_0 to x_1 , then slower jumps when you come closer to x^*

V6: Learning rate (γ)

We kept γ as constant in the update equation.

Learning rate = step size.

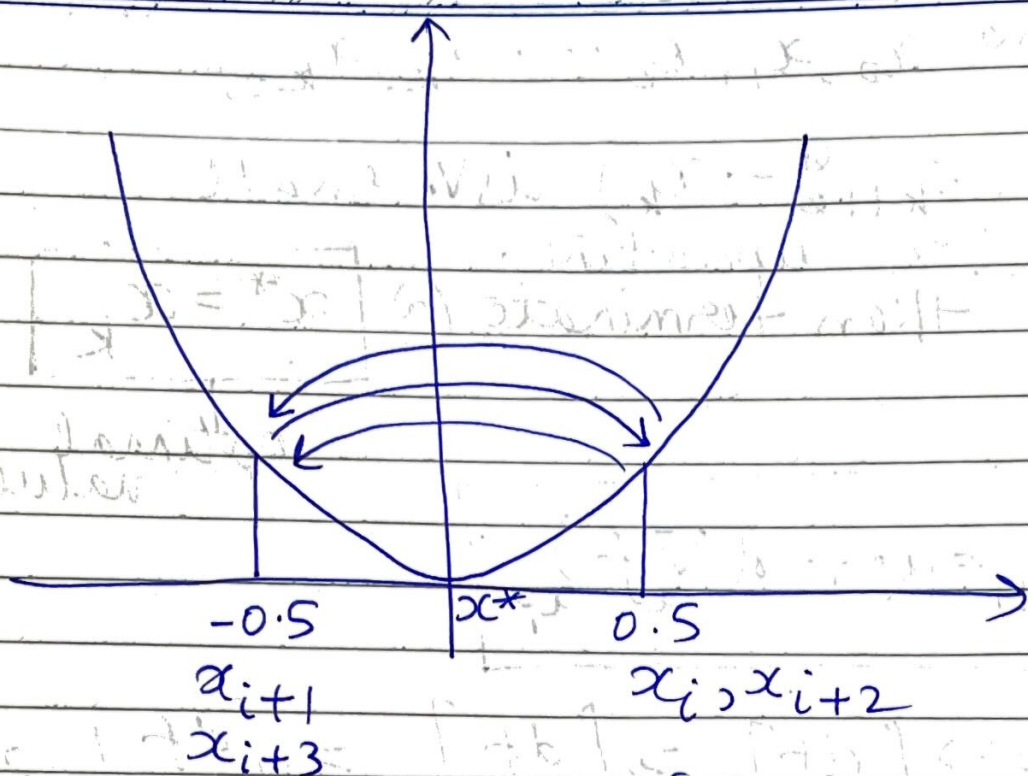
But it could create problems:

Suppose $x_i = 0.5$, $\frac{df}{dx} = 2x$

Then $x_{i+1} = x_i - \gamma * \left[\frac{df}{dx} \right]_{x_i}$

Let $\gamma = 1$.

$$x_{i+1} = 0.5 - 1 * [2 * 0.5] \\ = -0.5$$



$$x_{i+2} = -0.5 - 1 * (2 * (-0.5))$$

$$= 0.5$$

So, I am oscillating b/w $x = 0.5$ and $x = -0.5$, $\therefore x$ is constant. and will never converge to x^* which is a problem.

So, remedy to oscillation problem:-
So, change γ with each iteration.
Reduce γ with each iteration.

So, γ becomes a function of your iteration.

$\gamma = h(i)$, where i is the iteration
st. $i \uparrow, \gamma \downarrow$

V7: Gradient descent for linear regression

$$W^* = \underset{W}{\operatorname{argmin}} \sum_{i=1}^n (y_i - \underbrace{W^T x_i}_{\substack{\text{no intercept} \\ W_0}})^2 \quad \text{no reg.}$$

How to apply grad. descent?

$$L(W) = \sum_{i=1}^n (y_i - W^T x_i)^2, \quad \langle x_i, y_i \rangle \rightarrow \Delta_{\text{train}}$$

, where, $x_i \rightarrow$ vector, $y_i =$ scalar

$$\nabla_W L = \sum_{i=1}^n \{ 2(y_i - W^T x_i)(-x_i) \}$$

Step 1: Pick a random vector W_0

Step 2: $W_1 = W_0 - \gamma \sum_{i=1}^n (-2x_i)(y_i - W_0^T x_i)$

Step 3: $W_2 = W_1 - \gamma \sum_{i=1}^n (-2x_i)(y_i - W_1^T x_i)$

Compute $W_0, W_1, W_2, \dots, W_k, W_{k+1}$

If $W_{k+1} - W_k$ is a vector of very small values.

$$W^* = W_k$$

$$W_j = W_{j-1} - \gamma \sum_{i=1}^n (-2x_i)(y_i - W_{j-1}^T x_i)$$

If n is large say $n = 1M$
 then computing summation, is
 very expensive. (whole dataset is computed)

for each iteration of w we have
 to iterate over all n (dataset).

And Grad. descent becomes very
 expensive ^{slow} and time consuming.

To improve problem of Grad. Descent, we do SGD;
V8: SGD Algorithm

SGD - Stochastic Gradient Descent.
 The most important optimization algo.

$$W_j = W_{j-1} - \gamma * \sum_{i=1}^k (-2x_i)(y_i - W_{j-1}^T x_i)$$

Replaced n with k , where
 $1 \leq k \leq n$ (pick random
 set of k points)

If you have GD where $n = 1M$
 you might need 100 iterations.
 and

For SGD if $k=10$, then you might need >100 iterations to converge.

For SGD, if $k=10$, you might need 1000 iterations to converge.

k = batch size in SGD.

(batch of random pts.)

it is called SGD.

often $k=1$ ~~but where $n \ll k$~~ it is called Batch SGD with batch size = 100

eg $n=1M$

$k=100$ (batch size)

You will get same result with SGD as with GD.

v9: Constrained Optimization & PCA

$$\text{PCA: } \max_u \frac{1}{n} \sum_{i=1}^n (u^T x_i)^2 \approx \max_x f(x)$$

s.t. $u^T u = 1$ s.t. $g(x) = c$

constraint

Objective function

General constrained problem:

$$x^* = \max_{x} f(x)$$

objective func.

s.t. $g(x) = c \leftarrow$ equality constraint
 $h(x) \geq d \leftarrow$ inequality

1990-1991

[illegible]

Figure 1

[Faint, illegible handwritten notes]

1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646, 2647, 2648, 2649, 2650, 2651, 2652, 2653, 2654, 2655, 2656, 2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2677, 2678, 2679, 26

Journal of Management Inquiry 18(6)br/>© The Author(s) 2009
Reprints and permissions:
<http://www.sagepub.com/journalsPermissions.nav>

As per the investigation conducted
between A/C and P/S

Take along with you - any items that you need to take with you

2000

Figure 1. The effect of the concentration of the *Agrobacterium* suspension on the transformation efficiency of *Agrobacterium* strains.

Figure 1. (a) Schematic diagram of the experimental setup. (b) Photograph of the experimental setup. (c) Photograph of the experimental setup. (d) Photograph of the experimental setup.

[illegible]

Now Solving PCA using Lagrangian multiplier:-

Find u that maximizes .

$$\max_u u^T S u \quad \text{Constant}$$

s.t. $u^T u = 1$

$$S = \frac{1}{n} \sum_{i=1}^n x_i x_i^T$$

Covariance matrix

~~$\max_u u^T S u$, s.t. $u^T u = 1$~~

~~$S \in \mathbb{R}^{n \times n}$~~

$$L(u, \lambda) = u^T S u - \lambda (u^T u - 1)$$

$$\frac{\partial L}{\partial u} = 0 \Rightarrow \frac{\partial (u^T S u - \lambda u^T u - \lambda)}{\partial u} = 0$$

$$\Rightarrow 2(Su - \lambda u) = 0$$

$$\Rightarrow \boxed{Su = \lambda u}$$

defn of eigen vector & value

eigen vector of S

eigen value of S

eigen vector

V10: Logistic regression formulation revisited

$$W^* = \underset{\text{over } W}{\operatorname{argmin}} (\text{logistic loss}) + \underbrace{\lambda(W^T W)}_{\text{reg.}}$$

As a constrained opt. problem:

$$W^* = \underset{W}{\operatorname{argmin}} (\text{logistic loss}) \quad \xrightarrow{\text{obj. func.}}$$

$$\text{s.t. } W^T W = 1$$

$\uparrow$ equality constraint

$$L = \text{logistic loss} - \lambda(1 - W^T W)$$

$$= \text{logistic loss} - \lambda + \lambda W^T W$$

λ Lagrange multiplier

reg. can be thought as an equality constraint.

VII: Why L1 regularization creates sparsity

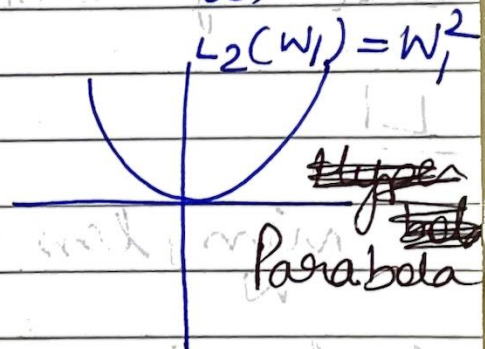
L1 reg. creates sparsity in weight vector as compared to L2 reg.
Sparse means in W vector more W components are zero.

$$\min_W \text{loss} + \lambda \|W\|_2^2$$

Only difference in both L2 & L1 is $\|W\|_2^2$ and $\|W\|_1$. Loss and λ are constant.
Ignoring other terms and looking at W as other terms are constant.
So, $\min_{W_1, W_2, \dots, W_d} (W_1^2 + W_2^2 + \dots + W_d^2)$

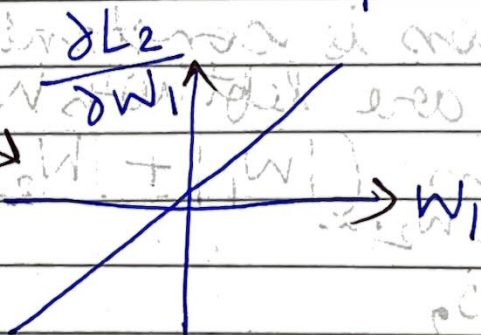
Now, $\min_{W_1} (W_1^2) = L_2(W_1)$

$$L_2(W_1) = W_1^2$$



Derivative func.

$$\text{der } \frac{\partial L_2}{\partial W_1} = 2 * W_1$$



QD $\rightarrow$

$$(W_1)_0, (W_1)_1, (W_1)_2, \dots, (W_1)_K$$

0.080
0.001
0.049

dear

iteration

iteration

So, $(w_i)_{j+1} = (w_i)_j - \gamma \left[\frac{\partial L_2}{\partial w_i} \right]_{w_i}$

$\Rightarrow (w_i)_{j+1} = (w_i)_j - \gamma \cdot 2w_i$

Say w_i is +ve at j .
Let $\gamma = 0.01$, $w_{ij} = 0.05$

$\downarrow$ $2 \times \gamma \times w_{ij} = 0.001$ (Extremely small)
So, $0.05 - 0.001 = 0.049$

$\Rightarrow$ L2 regularization doesn't change the value of w_i from one iteration to another $\because$ slope is slower. (back page)

$\Rightarrow$ There is less chance of $w_i = 0$ at the end of k -iterations.

L1

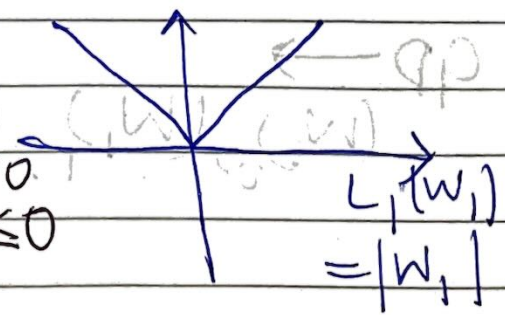
$\min_w \text{loss} + \lambda ||w||_1$

As loss is constant and so is λ .
We are left with w because other terms are constant

$\min_{w_1, w_2, \dots} (|w_1| + |w_2| + |w_3| + \dots + |w_d|)$

Now,
 $\min_{w_1} (|w_1|) = L_1(w_1)$

$L_1(w_1) = |w_1| = \begin{cases} w_1 & \text{if } w_1 > 0 \\ -w_1 & \text{if } w_1 \leq 0 \end{cases}$



Derivative function

$$\frac{\partial L_1}{\partial W_1} = \begin{cases} +1, & W_1 > 0 \\ -1, & W_1 < 0 \end{cases}$$

$\partial L_1 / \partial W_1$



$$(W_1)_{j+1} = (W_1)_j - \gamma \left(\frac{\partial L_1}{\partial W_1} \right) w_{ij}$$

Say W_1 is +ve at j

as $W_1 > 0$
it is +1

$$\text{Then } (W_1)_{j+1} = (W_1)_j - (\gamma) * 1$$

$\gamma = 0.01$ Small value

$$\gamma * 1 = 0.01$$

$$\Rightarrow 0.05 - 0.01 = 0.04$$

~~So converge~~

$\Rightarrow$ L1 regularizer continues to constantly reduce W_1 towards $W_1^* (= 0)$
∵ gradient/slope is constant.

$\Rightarrow$ chance that $W_1 = 0$ are more at the end of k -iterations.

Read this topic by reading L2 and L1 simultaneous comparison.

To get to the derivation of this we need to consider the loss function as well.